

Desarrollo de Software en Sistemas Distribuidos

Cátedra: Azcurra – Amaro - Tomczak.

Cuatrimestre: Segundo

Año: 2025

TRABAJO PRÁCTICO

RPC - Empuje solidario

Grupo 5: Integrantes

Nombre	Martin Bombara	Daniel Geier	Sebastián Marioni	Luciano Moliterno
Tareas Realizadas	- Dockerizacion Kafka - Creación de consumers en servicios - Creación producer de transferencia donación	- Creación consumer para el tópico ofrecer-donaciones - Creación producer para el tópico ofrecer-donaciones - Persistencia en la base de datos en una nueva tabla "ofertas_externas"	Creación de las vistas y de los servicios para solicitar, transferir y ofertar en el frontend Pendiente por realizar: ● Producir los mensajes mediante los formularios de las vistas (Conectar el front con el back) ● Refactorizar código	- view login - view registro usuarios - roles usuarios - interfaz proto usuarios - schema db users - db users - login de un usuario se agregó en la rama inventario-grpc-eventos-gateway-darkmode: nuevo microservicio inventario + gateway flask (expone inventario) + docker compose actualizado para levantar inventario y conectar el gateway por gRPC ----- cree las ramas: Gestión de Eventos Externos (Puntos 5 y 6) + Adhesión de Voluntarios a Eventos (Punto 7)



DNI	40812039	40229425	42281606	40238958
-----	----------	----------	----------	----------

Repositorio Github

<https://github.com/DanyGeier/Grupo-TN-G>

TP GRPC (Completo)

🚩 TP 1 COMPLETO

Pendientes

- *Frontend para visualización de eventos externos (esta en la rama frontend-mensajería, debido a que aun no esta implementado correctamente)*
- *Lógica de negocio para mensajes en Kafka relacionados a Eventos (Puntos 5, 6 y 7)*
- *Flujo de mensajes en el archivo inventario.proto para ofrecer-donaciones.*
- *Tópico baja-solicitud-donación (en desarrollo en branch: feat/punt-4-baja-solicitud)*

Estrategia de resolución

Tecnologías utilizadas

- Frontend WEB: **TS con React + Vite**
- Cliente Rest <-> gRPC : **Python**
- Servicios backend (Usuarios, Eventos, Donaciones) : **Java con Spring Boot**
- DB Servicio Usuarios y Donaciones: **MySQL**
- DB Servicio Eventos: **MongoDB**
- SMTP Envío de emails: **Mailpit**
- Broker / plataforma para envío de eventos : **Apache Kafka**
- Virtualizacion / Contenedores: **Docker**
- Otras herramientas de desarrollo:

- Postman
- Bruno
- Trello

Descripción de la resolución:

Arquitectura y división de servicios

Con el agregado del manejo de eventos por Apache kafka se incrementó la complejidad de la solución. Lo que planteamos fue la utilización de esta herramienta junto con KRaft quien reemplaza a Zookeeper para el manejo y coordinación de clusters y trae como ventaja una mayor simplicidad para su utilización además de consumir menos recursos de memoria. También viene integrado a kafka y no precisa de una instancia con una herramienta externa.

Incorporamos el uso de Kafka UI para poder ver el estado y situación de cada tópico además de permitirnos producir mensajes con facilidad

Cada evento producido por nuestro frontend tiene que pasar por el cliente de Python que expone el endpoint rest y se comunica por gRPC con cada uno de los microservicios. Cada uno de los servicios que requiere los servicios de Kafka expone N consumers y producers dependiendo lo que necesite obtener o producir, teniendo en cuenta que los dividimos a 1 por tópico (1 consumer no escucha más de un tópico, para escuchar 2 tendrían que ser 2 consumers). Ambos son parte de los servicios y no se crearon aplicaciones nuevas para los mismos.

Los eventos externos son simulados por Kafka UI y recibidos por cada consumer registrado. Los eventos internos hacen el camino front -> cliente python rest /grpc -> Servicio -> Producer.

Seguimos una “Event Driven Architecture + Event sourcing” para pensar el diseño de la solución empleada.

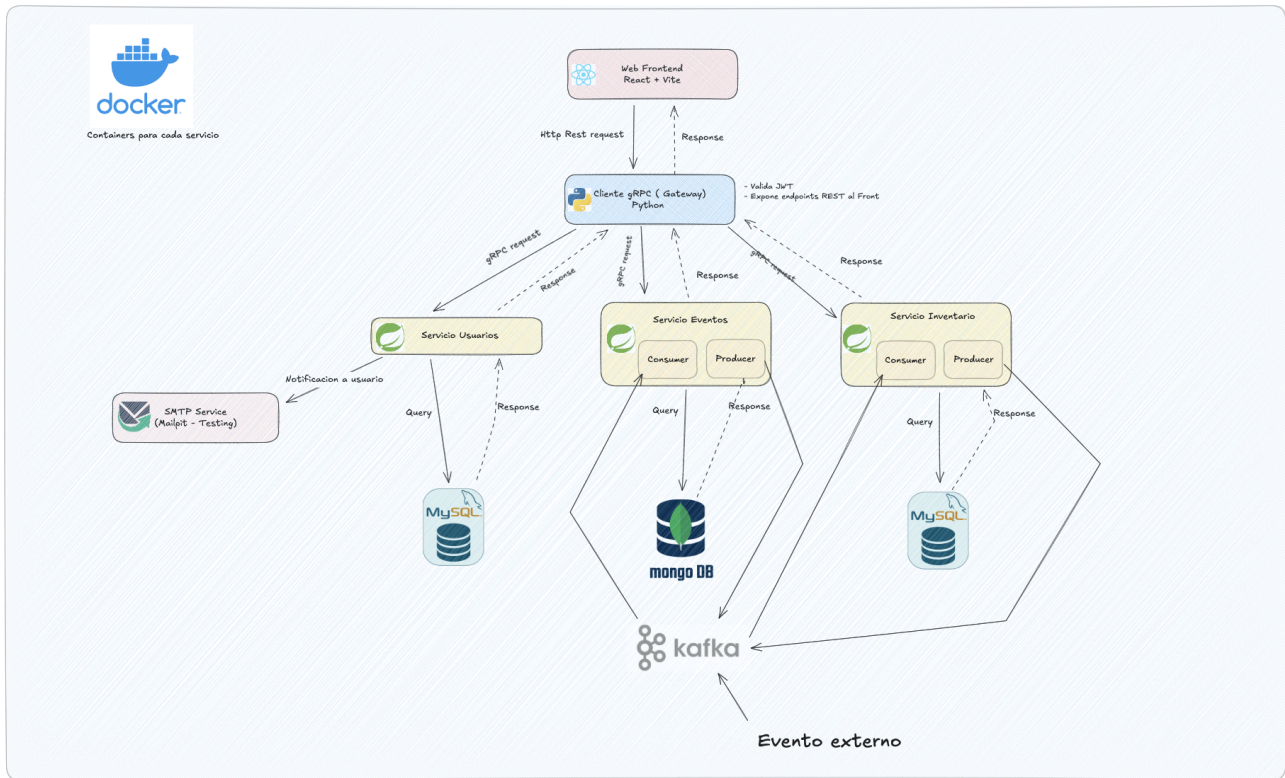
Por cada evento que escuchamos a través de los consumers guardamos o actualizamos un registro en nuestras DB en consecuencia, para mantener el estado interno de la aplicación basándonos en los eventos externos que pueden afectarla.

Organización de trabajo/división de actividades

Utilizamos Trello para gestionar tareas y coordinar al equipo. Cada proyecto se organiza en tableros con listas que reflejan el estado de las actividades. Las tarjetas representan tareas individuales, y contienen a los miembros responsables de realizarlas.

<https://trello.com/b/yznApxNZ/temp-tp-distribuidos>

Diagramas



Problemas/Errores

- Entendimiento de Kafka al principio por su complejidad
- Manejo de memoria y espacio con Docker al tener tantos containers en simultáneo
- Diseño de la solución en cuanto a buenas prácticas para el manejo de eventos

Pruebas

Opción 1: Descripción de las pruebas realizadas con las correspondientes capturas de pantalla.

Opción 2: En caso que prefieran hacerlo en video: enlace al video (deberá ser un video con voz, especificando qué pruebas se están ejecutando)

Video de pruebas:



Video 1 (Arquitectura y diseño) : 🎥 Grabación de pantalla 2025-10-12 a la(s) 8.12.07 p. m..mov

Video 2 (Funcionamiento en vivo): 🎥 Grabación de pantalla 2025-10-12 a la(s) 8.15.49 p. m..mov

☰ Capturas Frontend